

The SRv6 Core

A tenant provisioning portal, and a looking glass that remembers



VMs, one model

VyOS SRv6 core (uSID f3216) · IS-IS level-2 · BGP L3VPN with End.DT46 · two route reflectors

Nautobot source of truth · Network-as-Code rendering · Robot Framework validation · Prometheus and Grafana

What the core runs

Every number below is read from the routers or from the collector's own history — none of it is typed in.

2 / 8

tenants, sites

one VRF per tenant on every PE and CE; tenants never meet

76

prefixes in the core

152 paths in the VPNv4 / VPNv6 table, across two address families

100

tests per change

twelve Robot Framework suites against the running lab

~6 min

to add a tenant

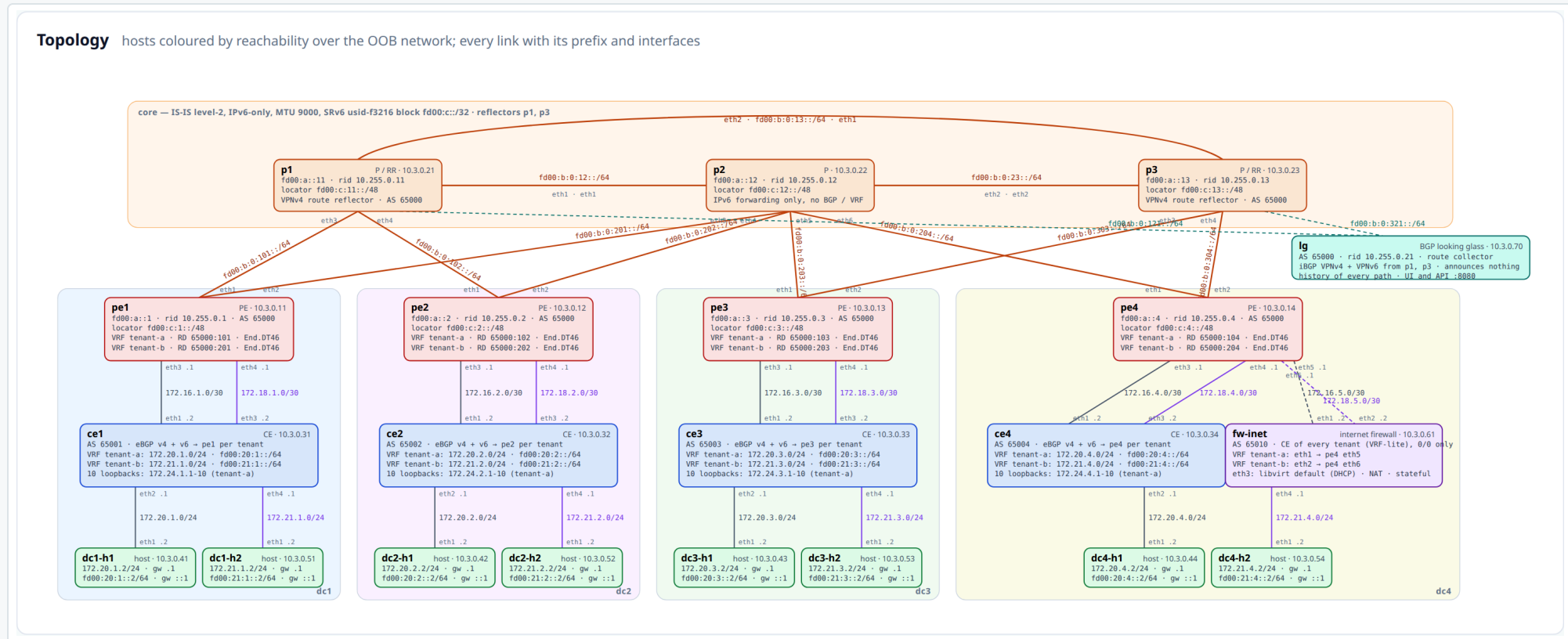
four sites, four host VMs, the CEs re-wired, Nautobot seeded

Two halves of the same model.

The portal changes the service: a tenant is described on a form, allocated from the live lab, and built by one run that ends in the test suites. The looking glass never changes anything — it sits in the core as a passive BGP listener, reads every router besides, and keeps what it saw, so any prefix can be answered for now and for any moment in the last thirty days.

The testbed

An IS-IS level-2, IPv6-only core carrying SRv6 uSID f3216; four data centres behind four PEs; p1 and p3 reflect, and the collector (top right) listens to both.



The portal's own drawing, from the model: every link with its prefix and interfaces, every VRF with its route distinguisher, hosts coloured by live reachability

The picture, the routers' configuration, the Nautobot objects and the test fixtures all come from one file — lab.conf plus the intent. Nothing on this slide is drawn by hand, and Nautobot must render the same thing or the build fails.

Part one

Provisioning a tenant

The portal is the only place the service is changed: a form states the intent, the allocator answers from the running lab, and a single run builds it — VMs, wiring, configuration, Nautobot and the tests.

<http://192.168.50.231:8091>

Tenants, with the live state beside them

One card per tenant: its sites, the attachment circuits, the CEs, the hosts — joined with what the PEs actually report.

Srv6 Tenant Provisioning Portal

VyOS SRv6 core · IS-IS + BGP L3VPN (End.DT4) · Nautobot source of truth · Robot Framework validation

Tenants

Steering

Runs

API

Looking glass

Nautobot: [open](#)

[dark](#)

2

Tenants

8

Sites

8 / 8

Hosts reachable

4

PEs

p1, p3

Route reflectors

0

Steering policies

Tenants

one VRF per tenant on every PE and CE; tenants never meet

Refresh live state

Add tenant

tenant-a

up

table 100 · RT 65000:100 · [VRF](#) · [tenant](#) · [prefixes](#)

Throughput

Add site

Remove

Site	PE	RD	Attachment circuit	CE	LAN	Host	eBGP CE -- PE	VRF routes (SRv6)	End.DT4 SID	
dc1	pe1	eth3	65000:101	172.16.1.0/30 fd00:16:1::/64	ce1 eth1 / eth2	172.20.1.0/24 fd00:20:1::/64	dc1-h1 up 172.20.1.2 · fd00:20:1::2 · oob 10.3.0.41	Established v6 up 11 pfx	49 (25)	fd00:c:1:e001::
dc2	pe2	eth3	65000:102	172.16.2.0/30 fd00:16:2::/64	ce2 eth1 / eth2	172.20.2.0/24 fd00:20:2::/64	dc2-h1 up 172.20.2.2 · fd00:20:2::2 · oob 10.3.0.42	Established v6 up 11 pfx	49 (25)	fd00:c:2:e002::
dc3	pe3	eth3	65000:103	172.16.3.0/30 fd00:16:3::/64	ce3 eth1 / eth2	172.20.3.0/24 fd00:20:3::/64	dc3-h1 up 172.20.3.2 · fd00:20:3::2 · oob 10.3.0.43	Established v6 up 11 pfx	49 (24)	fd00:c:3:e001::
dc4	pe4	eth3	65000:104	172.16.4.0/30 fd00:16:4::/64	ce4 eth1 / eth2	172.20.4.0/24 fd00:20:4::/64	dc4-h1 up 172.20.4.2 · fd00:20:4::2 · oob 10.3.0.44	Established v6 up 11 pfx	49 (24)	fd00:c:4:e001::

tenant-b

up

table 200 · RT 65000:200 · [VRF](#) · [tenant](#) · [prefixes](#)

Throughput

Add site

Remove

Site	PE	RD	Attachment circuit	CE	LAN	Host	eBGP CE -- PE	VRF routes (SRv6)	End.DT4 SID	
dc1	pe1	eth4	65000:201	172.18.1.0/30 fd00:18:1::/64	ce1 eth3 / eth4	172.21.1.0/24 fd00:21:1::/64	dc1-h2 up 172.21.1.2 · fd00:21:1::2 · oob 10.3.0.51	Established v6 up 1 pfx	9 (5)	fd00:c:1:e000::
dc2	pe2	eth4	65000:202	172.18.2.0/30 fd00:18:2::/64	ce2 eth3 / eth4	172.21.2.0/24 fd00:21:2::/64	dc2-h2 up 172.21.2.2 · fd00:21:2::2 · oob 10.3.0.52	Established v6 up 1 pfx	9 (5)	fd00:c:2:e001::
dc3	pe3	eth4	65000:203	172.18.3.0/30 fd00:18:3::/64	ce3 eth3 / eth4	172.21.3.0/24 fd00:21:3::/64	dc3-h2 up 172.21.3.2 · fd00:21:3::2 · oob 10.3.0.53	Established v6 up 1 pfx	9 (4)	fd00:c:3:e000::
dc4	pe4	eth4	65000:204	172.18.4.0/30 fd00:18:4::/64	ce4 eth3 / eth4	172.21.4.0/24 fd00:21:4::/64	dc4-h2 up 172.21.4.2 · fd00:21:4::2 · oob 10.3.0.54	Established v6 up 1 pfx	9 (4)	fd00:c:4:e000::

Internet breakout

up

fw-inet (AS 65010) is a CE of every tenant on pe4 — VRF-lite, one circuit per tenant — announcing only a default route; NAT out of eth3 on the host's libvirt network default; stateful policy tenant → Internet

Model and reality, in one row

The left of each row is the model (PE port, RD, attachment circuit, LAN, host); the right is read live from the PE.

A session that is not Established, or a host that does not answer, shows here first — there is no separate health page to remember.

VRF, tenant and prefixes link straight into Nautobot, so the source of truth is one click from the operational view.

Each row carries the eBGP session to the CE, the prefixes received, the VRF route count and the tenant's End.DT4 SID

Add a tenant — who it is, and where

The wizard opens with everything it can work out already filled in: the next free name, kernel table and route target.

Add a tenant

Tenant name: tenant-c Kernel table: 300 Route target: 65000:300

Description: tenant-c: L3VPN over SRv6 (End.DT4)

Sites (one CE VRF, one attachment circuit and one host per site):
☒ dc1 ☒ dc2 ☒ dc3 ☐ dc4

Everything else is allocated for you: the attachment circuit /30s and LAN /24s from the tenant's block, the next free PE and CE ports, and an Alpine host per site (name, OOB address, console, node index).

Back Next Cancel

Step 1 — identity and the sites the tenant will have

Add a tenant

Tenant name: tenant-c Kernel table: 300 Route target: 65000:300

Description: tenant-c: L3VPN over SRv6 (End.DT4)

Sites (one CE VRF, one attachment circuit and one host per site):
☒ dc1 ☒ dc2 ☒ dc3 ☒ dc4

Everything else is allocated for you: the attachment circuit /30s and LAN /24s from the tenant's block, the next free PE and CE ports, and an Alpine host per site (name, OOB address, console, node index).

dc4: no free ports on pe4

Back Next Cancel

The allocator answers from the running lab, so it can say no

Left: tenant-c, kernel table 300, route target 65000:300 — the next free values, and a site in each of the four data centres.

Right: pe4 has no free ports left, so a fourth site is refused before anything is built rather than half-way through the run. Drop that site and the wizard goes on — the allocation is checked against the lab, not against a copy of it.

2 Add a tenant — the allocation

Attachment circuit /30s and LAN /24s from the tenant's blocks, the next free PE and CE ports, and a host per site.

Add a tenant

dc1 PE pe1, CE ce1

PE port
eth5

CE port → PE
eth5

CE LAN port
eth6

Attachment circuit /30
172.22.1.0/30

LAN /24
172.40.1.0/24

Host name
dc1-h3

Host OOB address
10.3.0.60

Console port
5322

Node index
22

dc2 PE pe2, CE ce2

PE port
eth5

CE port → PE
eth5

CE LAN port
eth6

Attachment circuit /30
172.22.2.0/30

LAN /24
172.40.2.0/24

Host name
dc2-h3

Host OOB address
10.3.0.62

Console port
5323

Node index
23

dc3 PE pe3, CE ce3

PE port
eth5

CE port → PE
eth5

CE LAN port
eth6

Attachment circuit /30
172.22.3.0/30

LAN /24
172.40.3.0/24

Host name
dc3-h3

Host OOB address
10.3.0.63

Console port
5324

Node index
24

Edit anything that should differ; the review re-validates against the running lab (free ports, unused prefixes, unique names).

Back

Next

Cancel

Allocation, not arithmetic

Addresses come from the tenant's own blocks — 172.(16+i) for the attachment circuits, 172.(20+i) for the LANs — so a new tenant never collides with an old one.

The host is part of the tenant: name, OOB address, console port and node index are allocated with everything else, and the VM is built by the run.

Edit anything. The next step re-checks free ports, unused prefixes and unique names against the lab as it is now.

Step 2 — every value suggested and editable; the review re-validates whatever you change

3

Add a tenant — review, then deploy

The last step is the whole change in one table: what each PE gets, what each CE gets, and the host behind it.

Add a tenant

tenant-c — kernel table 300, route target 65000:300; RD 65000:300+PE# on each PE

Site	PE side	CE side (VRF tenant-c)	Host	What is configured
dc1	pe1 eth5 172.22.1.1/30 VRF tenant-c, eBGP to the CE, sid vpn export auto	ce1 eth5 172.22.1.2/30, eth6 172.40.1.1/24 eBGP to the PE, announces 172.40.1.0/24	dc1-h3 172.40.1.2 (gw .1) Alpine, OOB 10.3.0.60, console 5322	PE: VRF + interface + BGP · CE: VRF + 2 interfaces + BGP (re-wired VM) · host VM · Nautobot: tenant, VRF, RT, prefixes, device, cables, peering
dc2	pe2 eth5 172.22.2.1/30 VRF tenant-c, eBGP to the CE, sid vpn export auto	ce2 eth5 172.22.2.2/30, eth6 172.40.2.1/24 eBGP to the PE, announces 172.40.2.0/24	dc2-h3 172.40.2.2 (gw .1) Alpine, OOB 10.3.0.62, console 5323	PE: VRF + interface + BGP · CE: VRF + 2 interfaces + BGP (re-wired VM) · host VM · Nautobot: tenant, VRF, RT, prefixes, device, cables, peering
dc3	pe3 eth5 172.22.3.1/30 VRF tenant-c, eBGP to the CE, sid vpn export auto	ce3 eth5 172.22.3.2/30, eth6 172.40.3.1/24 eBGP to the PE, announces 172.40.3.0/24	dc3-h3 172.40.3.2 (gw .1) Alpine, OOB 10.3.0.63, console 5324	PE: VRF + interface + BGP · CE: VRF + 2 interfaces + BGP (re-wired VM) · host VM · Nautobot: tenant, VRF, RT, prefixes, device, cables, peering

☒ run the Robot suites (VPN, end-to-end, Nautobot) at the end (~10 min)

What happens: lab.conf + day-0 configs → host VMs created and booted → the CEs at the chosen sites are re-wired (each reboots for ~60 s: that site's other tenants lose connectivity for that time) → configuration pushed over SSH to the PEs and CEs → Nautobot seeded → ping matrix + Nautobot-vs-lab.conf check → tests.

Back

Deploy

Cancel

One button, one run

lab.conf and the day-0 configs are written, the host VMs are created and booted, and the CEs at the chosen sites are re-wired.

Then the configuration is pushed to the PEs and CEs over SSH, Nautobot is seeded, and the run verifies itself: a ping matrix of the tenant's hosts and a Nautobot-versus-lab.conf render check.

A re-wired CE reboots for about a minute, and the review says so — that site's other tenants lose connectivity for that time.

Step 3 — per-site review, with the option to run the Robot suites at the end

Watch it run — and resume it if it fails

Every run is a list of steps with a streamed log and, at the end, the Robot report.

Runs

one at a time

Run the test suites

Started	Mode	Tenant	Status	Tests
9/16/2026, 11:36:56 PM	test		success	3/3
9/16/2026, 8:07:19 PM	remove	tenant-c	success	16/16
9/16/2026, 7:56:04 PM	remove	tenant-c	failed	
9/16/2026, 7:55:16 PM	remove	tenant-c	failed	
9/16/2026, 7:25:37 PM	tenant	tenant-c	success	52/52
9/16/2026, 7:09:14 PM	tenant	tenant-c	failed	

tenant tenant-c success 2026-09-16_19-25-37-0348

success Validate the allocation (from run 2026-09-16_19-09-14-f4cf) tenant-c: 4 site(s) — dc1 (dc1-h3, 172.22.1.0/24), dc2 (dc2-h3, 172.22.2.0/24), dc3 (dc3-h3, 172.22.3.0/24), dc4 (dc4-h3, 172.22.4.0/24)

success Register in lab.conf, render the day-0 configs (from run 2026-09-16_19-09-14-f4cf) hosts dc1-h3, dc2-h3, dc3-h3, dc4-h3; CEs ce1, ce2, ce3, ce4

success Create and boot the host VMs (from run 2026-09-16_19-09-14-f4cf) dc1-h3, dc2-h3, dc3-h3, dc4-h3

success Re-wire the CE VMs (new attachment circuit and LAN ports) (from run 2026-09-16_19-09-14-f4cf) ce1, ce2, ce3, ce4 re-wired and back

success Push the configuration to the PEs (from run 2026-09-16_19-09-14-f4cf) ce1, ce2, ce3, ce4, pe1, pe2, pe3, pe4 and CEs (SSH)

success Nautobot source of truth (seed) (from run 2026-09-16_19-09-14-f4cf) seeded

success Verify: tenant ping matrix, Nautobot rendering == lab.conf tenant reachable, Nautobot == lab.conf

success Robot Framework tests 52/52 passed

Tests: 52/52 passed · [report](#) · [log](#)

04 Vpn 5/5
05 End To End 4/4
09 Nautobot 7/7
01 Management 3/3
02 Underlay 3/3
03 Srv6 4/4
04 Vpn 5/5
05 End To End 4/4
06 Rr Redundancy 3/3
07 Steering 4/4
08 Failover 3/3
09 Nautobot 7/7

```
1001 h2 /home/dcantor/srv6-core/results/2026-09-16_19-26-26/configs/post-run/routes/dc1-h2.routes.txt (901 bytes)
[dc1-h3] /home/dcantor/srv6-core/results/2026-09-16_19-26-26/configs/post-run/routes/dc1-h3.routes.txt (901 bytes)
[dc2-h1] /home/dcantor/srv6-core/results/2026-09-16_19-26-26/configs/post-run/routes/dc2-h1.routes.txt (901 bytes)
[dc2-h2] /home/dcantor/srv6-core/results/2026-09-16_19-26-26/configs/post-run/routes/dc2-h2.routes.txt (901 bytes)
[dc2-h3] /home/dcantor/srv6-core/results/2026-09-16_19-26-26/configs/post-run/routes/dc2-h3.routes.txt (901 bytes)
[dc3-h1] /home/dcantor/srv6-core/results/2026-09-16_19-26-26/configs/post-run/routes/dc3-h1.routes.txt (901 bytes)
[dc3-h2] /home/dcantor/srv6-core/results/2026-09-16_19-26-26/configs/post-run/routes/dc3-h2.routes.txt (901 bytes)
```

The record

One run at a time, so two changes can never race on the same router.

A failed or interrupted run resumes from the failed step: the successful steps carry their earlier result forward, which is what the “from run ...” notes are.

The suites run inside the job, so the evidence belongs to the change rather than to a separate test cycle.

A real tenant-c run: eight steps, 52/52 tests — and every step marked “from run ...-f4cf” was inherited, not repeated

Steering: pin a prefix to a path

Explicit-path traffic engineering as a form — a source PE, a tenant prefix, and the P routers to cross, in order.

Explicit-path steering pin a tenant prefix on a PE to a segment list through chosen P routers — applied immediately

Source PE

Tenant

Prefix (a remote site LAN)

Path (P routers, in order; the first must be attached to the PE)

pe1

tenant-a

172.20.2.0/24 (dc2, behind pe2)

p1 p3

Apply

Policies present on the PEs

PE	Tenant	Prefix	Segment list	Out
The segment list is the End SID of every listed P router followed by the End.DT4 SID of the PE that owns the prefix (read live). Return traffic keeps the IGP shortest path. Suite 07 shows the SRH on the wire.				

The segment list is the End SID of each listed P router, then the End.DT4 SID of the PE that owns the prefix — read live and applied immediately. No policy is in place here.

Cheap, because SRv6

No per-hop state and no signalling: the ingress PE writes a segment list into the packet and every router in between only has to forward it.

One direction at a time

Return traffic keeps the IGP shortest path, so a policy is one-directional unless you add its mirror. Suite 07 shows the SRH on the wire.

And it is visible afterwards

The looking glass draws a steered prefix along the routers its policy named, and keeps drawing it that way for the window in which it was in force.

Part two

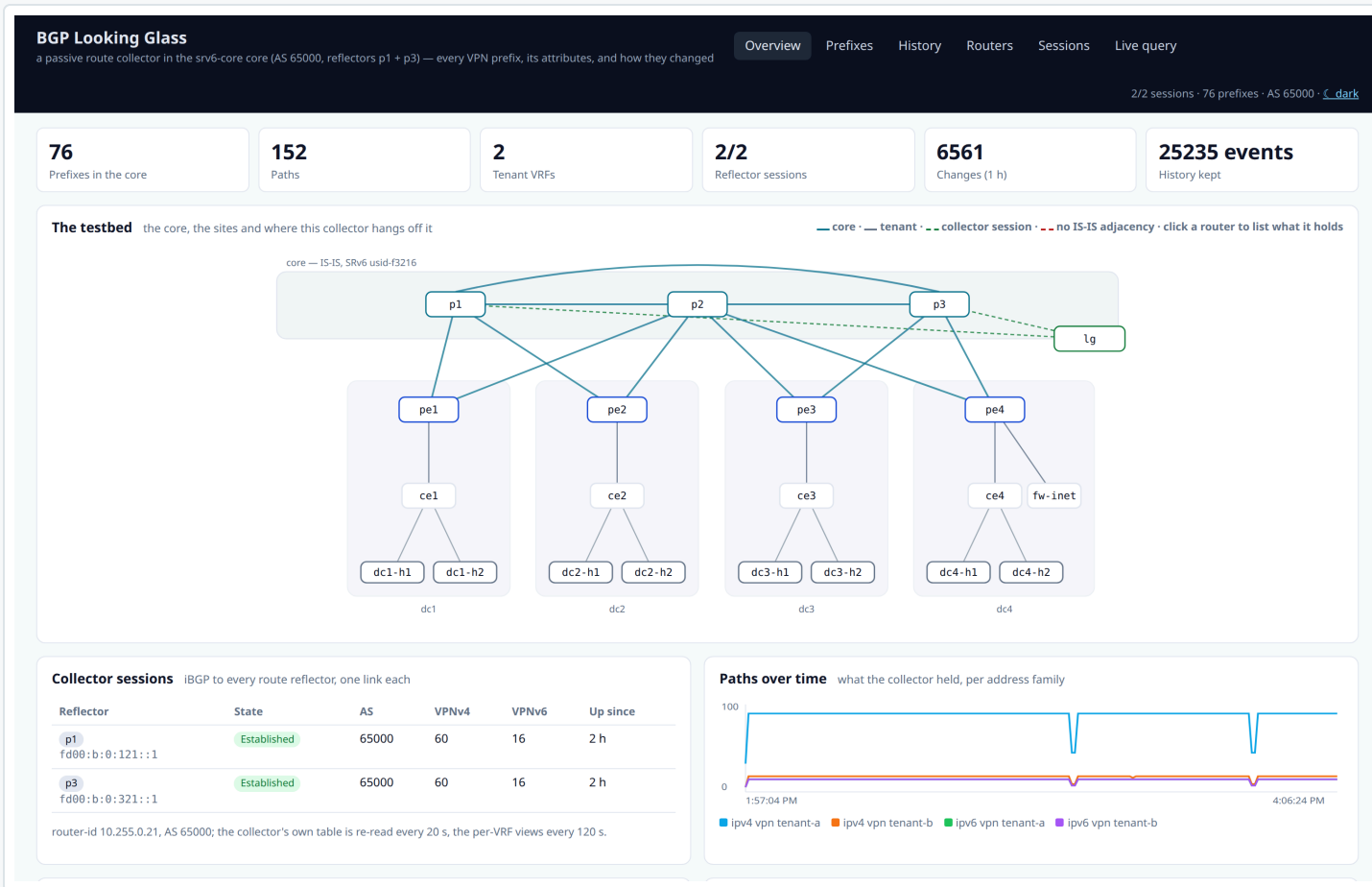
The looking glass

A twenty-first VM, Alpine and 512 MiB, sits in the core as a passive route collector: an iBGP session to each reflector, a direct read of every router besides, and a history of everything it has seen.

<http://192.168.50.231:8092>

Every VPN prefix in the core, and how it got there

The overview: what the collector holds, the sessions it holds it over, and the testbed drawn from the model.



Why a session, not screen-scraping

The collector receives the same update the reflector sent: route distinguisher, route targets, SRv6 SID and label, originator-id, cluster list, AS path, local preference.

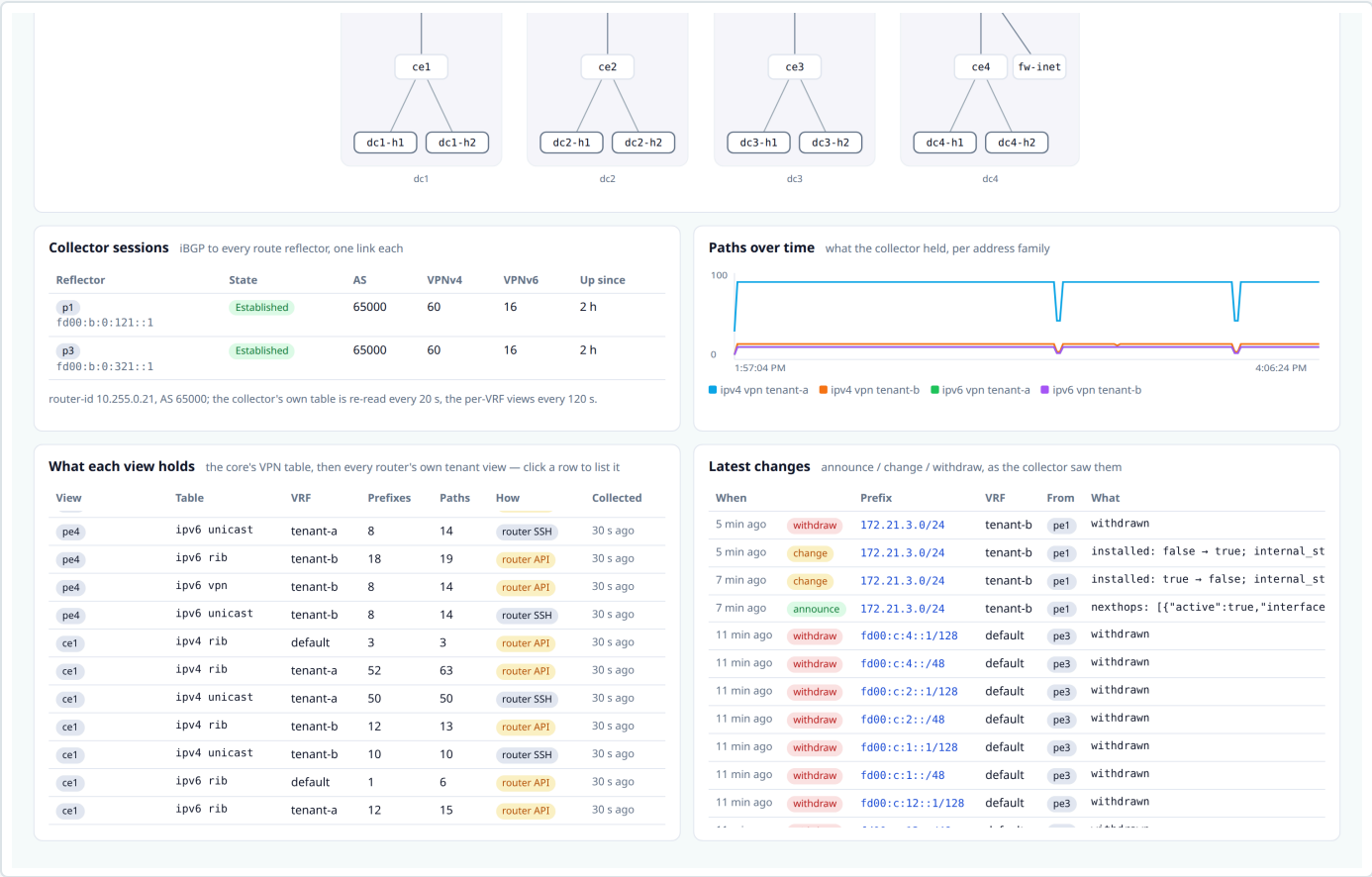
It announces nothing back — an outbound deny route-map says so explicitly, and the sessions show 0 sent.

The session negotiates extended next-hop capability, without which every VPNv4 route would look as though it came from the reflector instead of the PE that originated it.

76 prefixes, 152 paths, two reflector sessions — and 18,674 recorded changes behind them

Every row says how it was read

The core's VPN table is one answer. What a tenant actually has, and what a router will really do, are two others.



Three transports, one page

rr-session — the whole VPNv4 / VPNv6 table, as the reflectors handed it over.

router-api — each router's own HTTPS API: its RIB per VRF and family, its own VPN table, its IS-IS adjacencies.

router-ssh — the per-VRF BGP tables, where the API has no JSON form for the question.

The same prefix seen two ways is two answers, so the transport is a column, a filter and a pill — never hidden.

What each view holds, per router, VRF and family — with the transport it came over and when it was collected

Prefixes, filtered the way an operator asks

View, family, VRF, route distinguisher, origin AS, how it was read — and free text over prefix, next hop, SID, route target and AS path.

BGP Looking Glass

a passive route collector in the srv6-core core (AS 65000, reflectors p1 + p3) — every VPN prefix, its attributes, and how they changed

Overview

Prefixes

History

Routers

Sessions

Live query

2/2 sessions · 76 prefixes · AS 65000 · [dark](#)

Prefixes 3087 paths match

View

Address family

VRF

Route distinguisher

How it was read

Origin AS

Search (prefix, next hop, SID, RT, AS path)

every view

every family

every VRF

every RD

however it was

65001

172.20. / fd00:c:: / RT:65000:100

Search

Reset

The collector's view is the core's VPN table: one row per path a reflector handed over, with the attributes the PE originated.

	Prefix	VRF	RD	Next hop	From	AS path	Origin	Local pref	MED	SID	Route targets	Age	View	How
path	0.0.0.0/0 best	default	—	192.168.122.1 fw-inet	local	i	—	—	—	—	—	18 h	fw-inet	router API
path	0.0.0.0/0 best	tenant-a	—	172.16.1.1 ce1	local	i	—	—	—	—	—	18 h	ce1	router API
path	0.0.0.0/0 best	tenant-a	—	172.16.1.1	172.16.1.1	65000 65010	65010	—	0	—	RT:65000:100	26 h	ce1	router SSH
path	0.0.0.0/0 best	tenant-a	—	172.16.2.1 ce2	local	i	—	—	—	—	—	18 h	ce2	router API
path	0.0.0.0/0 best	tenant-a	—	172.16.2.1	172.16.2.1	65000 65010	65010	—	0	—	RT:65000:100	26 h	ce2	router SSH
path	0.0.0.0/0 best	tenant-a	—	172.16.3.1 ce3	local	i	—	—	—	—	—	18 h	ce3	router API
path	0.0.0.0/0 best	tenant-a	—	172.16.3.1	172.16.3.1	65000 65010	65010	—	0	—	RT:65000:100	26 h	ce3	router SSH
path	0.0.0.0/0 best	tenant-a	—	172.16.4.1 ce4	local	i	—	—	—	—	—	18 h	ce4	router API
path	0.0.0.0/0 best	tenant-a	—	172.16.4.1	172.16.4.1	65000 65010	65010	—	—	—	—	26 h	ce4	router SSH
path	0.0.0.0/0 best	tenant-a	65000:104	fd00:a::4 pe4	p1	65010	65010	100	0	fd00:c::	RT:65000:100	26 h	core	session
path	0.0.0.0/0 best	tenant-a	65000:104	fd00:a::4	p1	65010	65010	100	0	fd00:c::	RT:65000:100	26 h	core	session

Load more

showing 200 of 3087

The classic questions

“Who originates this?” — origin AS and originating node are columns, not something to derive from an AS path.

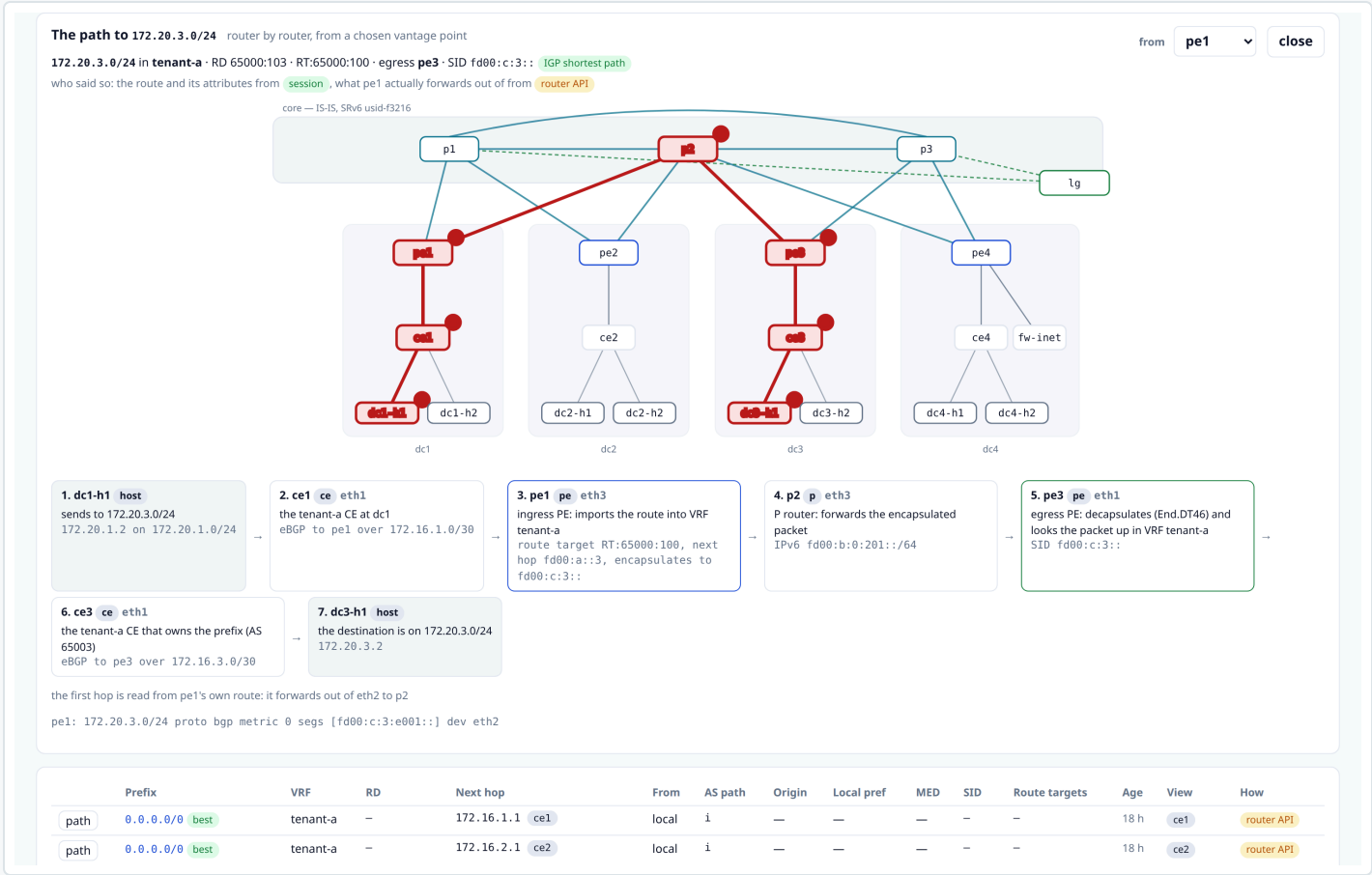
“Is it in the tenant's VRF, or only in the core?” — the view filter answers it directly.

“What SID does it carry?” — the End.DT46 SID is on the row, and the free-text box searches it.

Every row carries its attributes and a path button

The path, router by router

For one prefix from a chosen vantage point: the map with the path lit up, then a card per hop.



Two sources, one answer

The route and its attributes come from the session; what the router will actually do with the packet comes from its own RIB — and the card says which is which.

Equal-cost paths are named rather than silently collapsed into one line.

A steered prefix is drawn along the segment list the policy installed: the uSID carrier is unpacked back into the routers it names.

dc1-h1 → 172.20.3.0/24: the CE hand-off, pe1's import and SRv6 encapsulation, p2, pe3's End.DT46 decapsulation, ce3, the host

One prefix, every view at once

The core's VPN table, each reflector's and PE's own VPN table, each PE's and CE's VRF table, and every router's RIB.

the first hop is read from pe1's own route: it forwards out of eth2 to p2

pe1: 172.20.3.0/24 proto bgp metric 0 segs [fd00:c:3:e001::] dev eth2

The core's VPN table

as the reflectors handed it over

session

VRF	tenant-a	Route distinguisher	65000:103	Next hop	fd00:a::3	Advertised by	p1	Originating node	pe3	AS path	65003
Origin	IGP	Local pref	100	MED	0	Route targets	RT:65000:100	Originator-id	10.255.0.3	Cluster list	10.255.0.11
SRv6 SID	fd00:c:3::	Label	917520	Selection	best path	First seen	9/24/2026, 1:53:15 PM	Last change	9/25/2026, 3:49:04 PM	State	in the table
VRF	tenant-a	Route distinguisher	65000:103	Next hop	fd00:a::3	Advertised by	p3	Originating node	pe3	AS path	65003
Origin	IGP	Local pref	100	MED	0	Route targets	RT:65000:100	Originator-id	10.255.0.3	Cluster list	10.255.0.13
SRv6 SID	fd00:c:3::	Label	917520	Selection	multipath	First seen	9/24/2026, 1:53:15 PM	Last change	9/25/2026, 3:49:04 PM	State	in the table

On pe1

after import, in the tenant's VRF and its routing table and its own VPN table

router SSHrouter API

VRF	tenant-a	Next hop	fd00:a::3	Advertised by	local	Originating node	pe3	AS path	65003	Origin	IGP
Local pref	100	MED	0	Route targets	RT:65000:100	Originator-id	10.255.0.3	Cluster list	10.255.0.11	SRv6 SID	fd00:c:3::
Transposed SID	fd00:c:3:e001::	Label	917520	Imported from	65000:103	Selection	best path	First seen	9/24/2026, 1:53:16 PM	Last change	9/25/2026, 3:50:27 PM
State	in the table										
VRF	tenant-a	Next hop	fd00:a::3	Advertised by	local	Originating node	pe1	AS path	i (local)	SRv6 SID	segs
First seen	9/24/2026, 9:42:58 PM	Last change	9/25/2026, 3:56:24 PM	State	in the table						
VRF	tenant-a	Route distinguisher	65000:103	Next hop	fd00:a::3	Advertised by	p1	Originating node	pe3	AS path	65003
Origin	IGP	Local pref	100	MED	0	Weight	0	Selection	best path	First seen	9/24/2026, 9:47:17 PM
Last change	9/25/2026, 3:50:26 PM	State	in the table								
VRF	tenant-a	Next hop	fd00:a::3	Advertised by	local	Originating node	pe3	AS path	65003	Origin	IGP
Local pref	100	MED	0	Route targets	RT:65000:100	Originator-id	10.255.0.3	Cluster list	10.255.0.13	SRv6 SID	fd00:c:3::
Transposed SID	fd00:c:3:e001::	Label	917520	Imported from	65000:103	Selection	multipath	First seen	9/24/2026, 1:53:16 PM	Last change	9/25/2026, 3:50:27 PM
State	in the table										
VRF	tenant-a	Route distinguisher	65000:103	Next hop	fd00:a::3	Advertised by	p3	Originating node	pe3	AS path	65003
Origin	IGP	Local pref	100	MED	0	Weight	0	Selection	multipath	First seen	9/24/2026, 9:47:17 PM
Last change	9/25/2026, 3:50:26 PM	State	in the table								

One question, answered by every vantage point that has an opinion — each with its full attribute set and the transport it came over

Where answers differ

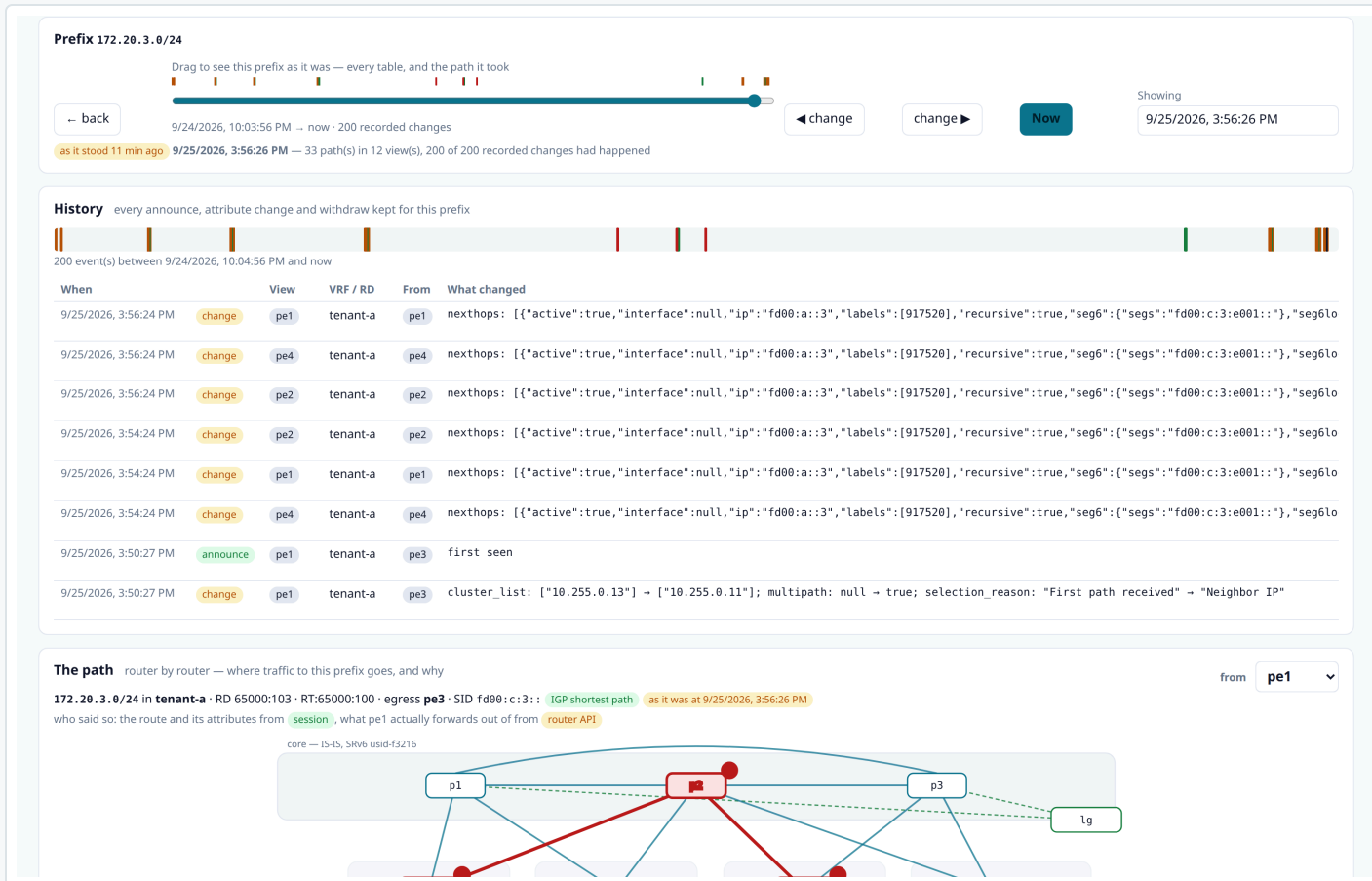
A prefix present in the core but missing from a VRF is an import problem; present in the VRF but not in the RIB is a selection problem. The page puts both side by side.

Attributes are shown whole: route targets, originator-id, cluster list, SID, label, selection reason.

Below it, the timeline of everything that has happened to this prefix.

And what it looked like an hour ago

Every collection is reconciled against what is stored, so the history is a log of announces, attribute changes and withdraws.



The point of keeping it

A change carries the fields that changed — local_pref: 100 → 200 — not just a new snapshot.

Drag the slider and every table on the page re-renders, and so does the path: the forwarding state comes out of the RIB history, not out of today's routers.

So a policy that was in place at 22:06 and is gone now still shows as the path it made. Thirty days are kept, pruned hourly.

The slider dragged back: 33 paths in 12 views as they stood at 13:57 — and the path diagram re-drawn for that moment

Everything it reads, and one question of your own

The Routers page is the collector's account of its own work; the Live query is the looking-glass button.

BGP Looking Glass

a passive route collector in the srv6-core core (AS 65000, reflectors p1 + p3) — every VPN prefix, its attributes, and how they changed

OverviewPrefixesHistoryRoutersSessionsLive query

2/2 sessions · 76 prefixes · AS 65000 · [dark](#)

The routers

every box the looking glass reads, and how it reads each part

router APIthe router's HTTPS API (vYOS 'service https api', JSON)

router SSHvtysh over SSH, for the views the API cannot express

sessionthe collector's own iBGP session to the route reflectors

Router	Role	Identity	Read through	Last	RIB	BGP (per VRF)	VPN table	IS-IS adjacencies
ce1 10.3.0.31	ce dc1	rid 172.20.1.1 · AS 65001	router API · router SSH	77 s ago 3.6 s	114	76	0	—
ce2 10.3.0.32	ce dc2	rid 172.20.2.1 · AS 65002	router API · router SSH	77 s ago 3.8 s	114	76	0	—
ce3 10.3.0.33	ce dc3	rid 172.20.3.1 · AS 65003	router API · router SSH	76 s ago 3.9 s	114	76	0	—
ce4 10.3.0.34	ce dc4	rid 172.20.4.1 · AS 65004	router API · router SSH	77 s ago 3.8 s	114	76	0	—
fw-inet 10.3.0.61	fw dc4	rid 10.3.0.61 · AS 65010	router API · router SSH	77 s ago 3.5 s	135	60	0	—
p1 10.3.0.21	p core	rid 10.255.0.11 · AS 65000 · fd00:a::11 · fd00:c::11::/48	router API	78 s ago 2.2 s	56	0	76	p2 · p3 · pe1 · pe2
p2 10.3.0.22	p core	fd00:a::12 · fd00:c::12::/48	router API	79 s ago 1.2 s	58	0	0	p1 · p3 · pe1 · pe2 · pe3 · pe4

What each router holds

by address family and VRF — click to list it

Router	Table	VRF	Prefixes	Paths	How
ce1	ipv4 rib	default	3	3	router API
ce1	ipv4 rib	tenant-a	52	63	router API
ce1	ipv4 rib	tenant-b	12	13	router API
ce1	ipv6 rib	default	1	6	router API
ce1	ipv6 rib	tenant-a	12	15	router API
ce1	ipv6 rib	tenant-b	12	14	router API
ce1	ipv4 unicast	tenant-a	50	50	router SSH
ce1	ipv4 unicast	tenant-b	10	10	router SSH
fw	ipv6 unicast	tenant-a	8	8	router SSH

Every box it reads: identity, transports, when, and how much it holds

Live query

a 'show' command straight on a router, or a ping from it

Device

pe1

Command

show bgp ipv4 vpn rd 65000:103 172.20.3.0/24

Run

Suggestions: show bgp vrf tenant-a ipv4 unicast · show bgp vrf tenant-a ipv6 unicast · show bgp ipv4 vpn · show bgp summary · show bgp ipv4 vpn detail · show ip route vrf tenant-a · show isis neighbor · show bgp neighbors

Only show —, ping <address> [vrf X] [count N] and traceroute <address> are accepted, and only on the lab's own devices.

pe1: show bgp ipv4 vpn rd 65000:103 172.20.3.0/24 (0.72 s)

```
BGP routing table entry for 65000:103:172.20.3.0/24, version 110
not allocated
Paths: (2 available, best #1)
  Not advertised to any peer
65002
  0.0.0.0 (metric 20) from fd00:a::11 (10.255.0.3)
    Origin IGP, metric 0, localpref 100, valid, internal, multipath, best (Neighbor IP)
    Extended Community: RT:65000:100
    Originator: 10.255.0.3, Cluster List: 10.255.0.11
    Remote Labels: 917520
    Remote SID: fd00:c::3::, sid structure=[32 16 16 0 16 48]
    Last update: Fri Sep 25 19:48:49 2026
65003
  0.0.0.0 (metric 20) from fd00:a::13 (10.255.0.3)
    Origin IGP, metric 0, localpref 100, valid, internal, multipath
    Extended Community: RT:65000:100
    Originator: 10.255.0.3, Cluster List: 10.255.0.13
    Remote Labels: 917520
    Remote SID: fd00:c::3::, sid structure=[32 16 16 0 16 48]
    Last update: Fri Sep 25 17:56:56 2026
```

A show command straight on a router — only show, ping and traceroute

Each router's own HTTPS API is rendered into its configuration like everything else (key srv6core-lab-looking-glass, reachable only from the collector and the lab host), so the same interface is open to anything else that wants it.

Numeric series — paths per VRF and family, churn, session state, collection health — go to /metrics, where Prometheus scrapes them for the dashboard's looking-glass row and three alerts.

18

What the two halves are for

Change it from one place

The portal allocates from the running lab, builds the tenant in one resumable run, and ends in the test suites — so the evidence belongs to the change.

See it as the network sees it

A passive BGP session gets the attributes the PEs originated; the routers' own APIs get what each box installed. Every row says which.

Ask about yesterday

The history is reconciled, not sampled: announces, attribute changes and withdraws, thirty days of them, with the path re-drawn for any moment in that window.

The portal is at <http://192.168.50.231:8091> and the looking glass at :8092, both with their REST API at /docs; the lab hub at :8088 lists every lab on the host. The full write-up is README.md and docs/srv6-walkthrough.pdf.